

CSCI 3485 Lab 3: CNNs and MLPs on FashionMNIST

Understanding the Performance of Convolutional Networks

Ryan Novitski

October 3, 2024

1 Introduction

In the realm of neural networks, Convolutional Neural Networks (CNNs) have become a staple for image classification tasks due to their ability to capture spatial hierarchies in data. CNNs sweep over an image's pixel data performing matrix operations between a section of the image and a kernel of predefined size. While performing this sweeping algorithm, the CNN is able to extract various features from an image that can be used to accurately classify objects with relatively high success rates.

The objective of this lab is to explore the performance differences between CNNs and MLPs using the FashionMNIST dataset and to analyze how architectural choices such as kernel size, number of filters, batch normalization, and dropout can potentially impact the accuracy and training time of CNNs. The FashionMNIST dataset consists of 28x28 grayscale images from 10 classes of various clothing items, providing a suitable platform for training these models with a challenging but not overly complex dataset.

2 Methodology

2.1 Overview

The lab consisted of three main experiments:

- Experiment 1 compares the performance of MLPs and CNNs. For this experiment, the CNN's filters were set to 32 and the kernel size set to 3 (as specified in the instructions)

- Experiment 2 varies the kernel sizes for the CNNs. The kernel sizes tested consisted of 2, 3, 5, 7, and 9.
- Experiment 3 varies the number of filters per layer for the CNNs.
- Additionally, the effects of batch normalization and dropout individually were tested and evaluated.

2.2 Design Decisions

The models were trained using the FashionMNIST dataset with the ADAM optimizer and a fixed learning rate of 1e-3. Each model was trained for 5 epochs. The code used for these experiments was written and executed using the PyTorch library. Model performance was evaluated in terms of both test accuracy and the number of parameters learned.

The CNN model was created as a CNN with generalized inputs to allow for the various parameters being tested in different experiments to be appropriately adjusted. The structure of the CNN from input to output consists as follows: 2 convolutional layers, 1 max-pool layer, 1 convolutional layer, 1 max-pool layer, 1 fully connected linear layer with an output of 200, and 1 fully connected linear layer with an output of 10 to account for the 10 FashionMNIST classes.

3 Results

3.1 Comparison between CNN and MLP Architectures

The first experiment compared the performances of a CNN with a kernel size of 3 and a 2-layer MLP with an input layer of 1000.

```
===== Comparing CNN and MLP Architectures =====
Training MLP...
Epoch [1/5], Loss: 0.4720
Epoch [2/5], Loss: 0.3667
Epoch [3/5], Loss: 0.3288
Epoch [4/5], Loss: 0.3027
Epoch [5/5], Loss: 0.2826
MLP Training Time: 158.17 seconds
Training CNN...
Epoch [1/5], Loss: 0.3747
Epoch [2/5], Loss: 0.2422
Epoch [3/5], Loss: 0.1981
Epoch [4/5], Loss: 0.1712
Epoch [5/5], Loss: 0.1439
CNN Training Time: 1826.51 seconds
Test Accuracy: 87.69%
Test Accuracy: 92.70%
```

Figure 1: Output of CNN and MLP

After training both models, the results showed that the CNN achieved a higher test accuracy of 92.70% compared to the MLP’s test accuracy of 87.69%. However, the MLP trained significantly faster, taking approximately 158.17 seconds, whereas the CNN required 1826.51 seconds to complete the same number of epochs. Although it is assumed CNN’s will usually take longer than an MLP, these results are possibly skewed due to the limitations of Google Colab. Because of the limited access to the site’s GPUs, the training time was often very inconsistent, even across tests of the same training scripts. However, even with this in mind, these results accurately illustrate the trade-off between computational efficiency and model complexity. CNNs require more time and resources due to their more complex architecture but yield better performance for image classification tasks like FashionMNIST.

3.2 Varying Kernel Sizes in CNN

In the second experiment, the effect of kernel size on the CNN’s performance was examined by testing kernel sizes of 2, 3, 5, 7, and 9. The test accuracies were then recorded for each kernel size.

The results show that smaller kernel sizes provided higher test accuracy, with the CNN using a kernel size of 3 yielding the highest accuracy of 92.77%. As the kernel size increased, the accuracy began to decline, with the largest kernel size of 9 achieving only 90.74% accuracy. These findings align with the intuition that smaller kernels capture more detailed spatial information, leading to better feature extraction, while larger kernels may miss out on finer patterns in the images.

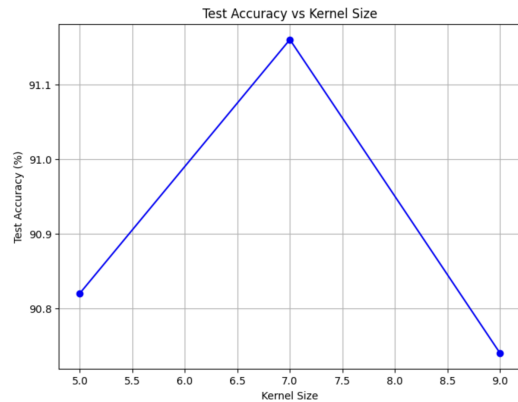


Figure 2: Accuracy vs. Kernel Size

In addition to test accuracy, training time was also observed. Although as previously mentioned it was difficult to get a true measurement of training time, larger kernel sizes generally took longer to train. This increase in training time with larger kernel sizes is expected, as larger kernels have more parameters to update during training, contributing to the longer time required for convergence. While small kernel sizes are more computationally efficient and yield higher accuracy, larger kernel sizes can become inefficient in both time and performance.

3.3 Varying Number of Filters in CNN

In the third experiment, the number of filters per convolutional layer were varied to observe its effect on the CNN’s performance. The filter sizes tested were 5, 10, 15, 20, and 25 filters per layer, with all other parameters kept constant.

The results of this experiment indicated that although the rate of improvement diminished after a certain point, increasing the number of filters generally improved accuracy. The CNN with 25 filters per layer achieved a test accuracy of 92.15%, which was slightly lower than the accuracy of 92.77% achieved in Experiment 2 with a kernel size of 3 and 32 filters.

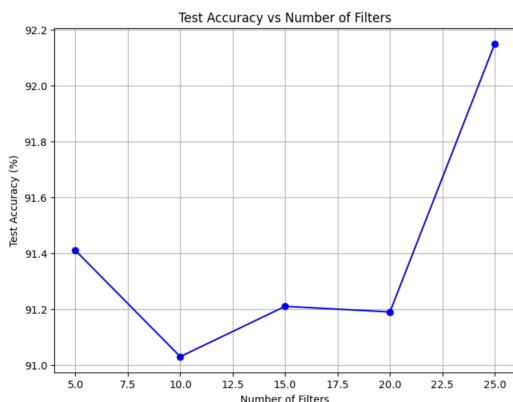


Figure 3: Accuracy vs. Filter Count

Although there would definitely be a benefit to testing a wider range of filters to provide a larger set of data to interpret, the results of this experiment indicate a positive trend between accuracy and filter count, suggesting that (at least at this range of options) a higher filter count can help increase a model's accuracy.

In terms of training time, CNNs with a larger number of filters required more time to converge, as expected. Because of this, it is important to keep in mind what is truly necessary for the problem at hand, as more filters can be beneficial, but excessive filters can become too computationally expensive. For the purpose of this lab, the 32 filter default seemed to be a solid balance between training time and desired accuracy.

3.4 Effect of Batch Normalization and Dropout

The last experiments were conducted to evaluate how batch normalization (BN) and dropout affect CNN

performance. A CNN model with batch normalization (but without dropout) achieved a test accuracy of 92.72%, which was slightly better than the base CNN model.

```
===== Testing CNN with Batch Normalization (No Dropout) =====
Training CNN with Batch Normalization...
Epoch [1/5], Loss: 0.3358
Epoch [2/5], Loss: 0.2110
Epoch [3/5], Loss: 0.1675
Epoch [4/5], Loss: 0.1370
Epoch [5/5], Loss: 0.1068
Test Accuracy: 92.72%
```

Figure 4: BN only output

On the other hand, when dropout was used (with no batch normalization), the test accuracy dropped slightly to 91.96%.

```
===== Testing CNN with Dropout (No Batch Normalization) =====
Training CNN with Dropout...
Epoch [1/5], Loss: 0.4304
Epoch [2/5], Loss: 0.2740
Epoch [3/5], Loss: 0.2234
Epoch [4/5], Loss: 0.1928
Epoch [5/5], Loss: 0.1665
Test Accuracy: 91.96%
```

Figure 5: Dropout only output

These results suggest that Batch Normalization has the potential to be more effective than dropout in improving performance for this particular dataset. Although there is not a drastic difference between the two model's accuracies, it would be interesting to expand this experiment to be tested on various configurations of CNNs to see if this trend holds.

4 Discussion/Conclusion

The experiments conducted in this lab highlight key aspects of how neural network architecture design decisions affect model performance on image classification tasks, particularly using the FashionMNIST dataset. By comparing CNNs and MLPs, the experiments reveal that CNNs generally outperform MLPs in terms of accuracy, owing to their ability to effectively extract classifying features within image data. The trade-off, however, is that CNNs require significantly longer training times due to their higher complexity.

The results from varying kernel sizes demonstrate that smaller kernel sizes tend to provide better

accuracy, with the optimal kernel size in this experiment being 3. Larger kernel sizes, while providing some benefit in capturing larger spatial patterns, often miss out on the finer details that are crucial for image recognition. This suggests that striking a balance in kernel size is necessary to optimize both accuracy and computational efficiency. In terms of filter count, increasing the number of filters per layer improved the performance up to a certain point, with diminishing returns observed at higher filter counts. This supports the notion that while larger models can yield better results, excessive parameters may lead to diminishing improvements relative to the increased training time.

Finally, the experiments on batch normalization and dropout highlight their respective impacts on model performance. Batch normalization was found to be slightly more effective than dropout in this instance, providing a marginal improvement in accuracy. However, the difference was not substantial, indicating that both techniques can be valuable depending on the specific use case. Further experimentation with different configurations would be necessary to make broader generalizations about the use of batch normalization and dropout.

In conclusion, this lab illustrates the importance of understanding the trade-offs involved in neural network architecture design. CNNs, while powerful, require more computational resources and careful tuning of parameters such as kernel size and filter count. Additionally, techniques like batch normalization and dropout can play critical roles in improving the robustness and accuracy of these models. For image classification tasks like FashionMNIST, careful attention to these design choices can lead to more efficient and accurate models, providing insight into how similar architectures may perform in other domains.

5 Code

```
1 import torch
2 import torch.optim as optim
3 import torch.nn as nn
4 import time
```

```
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Detect GPU
device = 'cuda' if torch.cuda.is_available()
else 'cpu'

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(28*28, 1000) #
        # Input layer
        self.fc2 = nn.Linear(1000, 10) #
        # Output layer (10 classes)

    def forward(self, x):
        x = x.view(-1, 28*28) # Flatten the
        # input
        x = torch.relu(self.fc1(x)) # Hidden
        # layer
        x = self.fc2(x) # Output layer
        return x

class GeneralizedCNN(nn.Module):
    """
    Generalized CNN model where parameters such
    as number of filters, kernel size, Batch
    Normalization,
    and Dropout can be adjusted for flexible
    experimentation. This allows for one model
    to be used for all
    desired experiments to reduce any
    redundancies.

    Args:
        num_filters (int): Number of filters
        for the first convolution layer
        kernel_size (int): Size of the
        convolution kernel.
        use_bn (bool): Whether to use Batch
        Normalization after convolution layers.
        use_dropout (bool): Whether to apply
        Dropout after certain layers.
        dropout_prob (float): Dropout
        probability to use if Dropout is applied.
    """
    def __init__(self, num_filters=32,
        kernel_size=3, use_bn=False,
        use_dropout=False, dropout_prob=0.5):
        super(GeneralizedCNN, self).__init__()
```

38		67	
39	# Padding to preserve spatial dimensions		
40	padding = (kernel_size - 1) // 2		
41		68	
42	# First Convolutional Block: Conv ->		
	(Optional BN) -> (Optional Dropout)	69	
43	self.conv1 = nn.Conv2d(1, num_filters,	70	
	kernel_size=kernel_size, padding=padding)	71	
44	self.bn1 = nn.BatchNorm2d(num_filters)		
	if use_bn else None	72	
45	self.dropout1 =	73	
	nn.Dropout(dropout_prob) if use_dropout		
	else None	74	
46		75	
47	# Second Convolutional Block: Conv ->	76	
	(Optional BN) -> Pooling	77	
48	self.conv2 = nn.Conv2d(num_filters,	78	
	num_filters * 2, kernel_size=kernel_size,	79	
	padding=padding)	80	
49	self.bn2 = nn.BatchNorm2d(num_filters * 2)	81	
	if use_bn else None		
50	self.pool = nn.MaxPool2d(2, 2) #	82	
	Reduces spatial dimensions by half	83	
51		84	
52	# Third Convolutional Block: Conv ->	85	
	(Optional BN) -> Pooling	86	
53	self.conv3 = nn.Conv2d(num_filters * 2,	87	
	num_filters * 4, kernel_size=kernel_size,	88	
	padding=padding)		
54	self.bn3 = nn.BatchNorm2d(num_filters * 4)	89	
	if use_bn else None	90	
55	self.pool2 = nn.MaxPool2d(2, 2) #	91	
	Further reduces spatial dimensions		
56		92	
57	# Dummy input to calculate the	93	
	flattened feature size	94	
58	self.example_input = torch.randn(1, 1,	95	
	28, 28)	96	
59	self.flatten_size =	97	
	self._get_flatten_size()	98	
60		99	
61	# Fully connected layers	100	
62	self.fc1 = nn.Linear(self.flatten_size,	101	
	200) # First fully connected layer		
63	self.fc2 = nn.Linear(200, 10) # Output	102	
	layer for 10 classes (FashionMNIST)		
64		103	
65	def _get_flatten_size(self):	104	
66	"""	105	

	Helper function to calculate the size
	of the flattened feature map after
	convolution and pooling.
	This is needed to set up the input size
	for the first fully connected layer.
	Returns:
	int: The total number of elements
	in the flattened feature map.
	"""
	x =
	torch.relu(self.conv1(self.example_input))
	if self.bn1: x = self.bn1(x)
	if self.dropout1: x = self.dropout1(x)
	x = self.pool(torch.relu(self.conv2(x)))
	if self.bn2: x = self.bn2(x)
	x = torch.relu(self.conv3(x))
	if self.bn3: x = self.bn3(x)
	x = self.pool2(x)
	return x.numel() # Return the number
	of elements in the flattened tensor
	def forward(self, x):
	"""
	Forward pass for the CNN model.
	Args:
	x (torch.Tensor): Input tensor
	(image data).
	Returns:
	torch.Tensor: The output logits for
	classification.
	"""
	x = torch.relu(self.conv1(x))
	if self.bn1: x = self.bn1(x)
	if self.dropout1: x = self.dropout1(x)
	x = self.pool(torch.relu(self.conv2(x)))
	if self.bn2: x = self.bn2(x)
	x = torch.relu(self.conv3(x))
	if self.bn3: x = self.bn3(x)
	x = self.pool2(x)
	x = x.view(-1, self.flatten_size) #
	Flatten before fully connected layers
	x = torch.relu(self.fc1(x)) # Fully
	connected layer
	x = self.fc2(x) # Output layer
	return x

```

106 def train_model(model, train_loader, optimizer, 143
107     loss_fn, n_epochs=5): 144
108     """ 145
109     Train the given model using the specified 146
110     DataLoader, optimizer, and loss function. 147
111     Args: 148
112     model (nn.Module): The model to train. 149
113     train_loader (DataLoader): DataLoader 150
114     for the training set. 151
115     optimizer (torch.optim.Optimizer): 152
116     Optimizer for updating model weights.
117     loss_fn (torch.nn.Module): Loss
118     function to compute training loss.
119     n_epochs (int): Number of training
120     epochs.
121     """
122     model.train()
123     train_loss = []
124     for epoch in range(n_epochs):
125         running_loss = 0.0
126         for inputs, labels in train_loader:
127             optimizer.zero_grad() # Zero the
128             gradients
129             outputs = model(inputs) # Forward
130             pass
131             loss = loss_fn(outputs, labels) #
132             Compute loss
133             loss.backward() # Backward pass
134             optimizer.step() # Update weights
135             running_loss += loss.item()
136             avg_loss = running_loss /
137             len(train_loader)
138             train_loss.append(avg_loss)
139             print(f"Epoch [{epoch+1}/{n_epochs}],
140             Loss: {avg_loss:.4f}")
141         return train_loss
142
143 @torch.no_grad()
144 def evaluate_model(model, test_loader):
145     """
146     Evaluate the given model on the test set.
147
148     Args:
149     model (nn.Module): The model to
150     evaluate.
151     test_loader (DataLoader): DataLoader
152     for the test set.
153     """
154     model.eval()

```

```

correct = 0
total = 0
for inputs, labels in test_loader:
    outputs = model(inputs) # Forward pass
    _, predicted = torch.max(outputs.data,
    1) # Get predicted labels
    total += labels.size(0)
    correct += (predicted ==
    labels).sum().item() # Count correct
    predictions
accuracy = 100 * correct / total
print(f"Test Accuracy: {accuracy:.2f}%")
return accuracy

```